



Aplicação prática de TDD

Neste módulo, vamos aplicar de forma prática tudo o que aprendemos sobre TDD, trazendo a teoria para situações do dia a dia em projetos reais. Começaremos analisando **exemplos de TDD em projetos**, observando como a escrita de testes desde o início influencia a estrutura do código e melhora a clareza das soluções. Vamos entender os passos concretos da criação de testes, implementação e refatoração em casos que refletem desafios comuns no desenvolvimento de software. Em seguida, estudaremos **padrões de TDD**, conhecendo abordagens recomendadas para organizar testes, facilitar a leitura do código e manter a evolução contínua dos sistemas de forma segura e sustentável. Depois, veremos como acontece a **integração do TDD com JUnit e Mockito**, aproveitando ambas as ferramentas para criar testes robustos, simulando comportamentos e controlando as dependências dos nossos componentes. Para concluir, falaremos sobre a **refatoração inicial baseada em testes**, reforçando como o TDD nos dá confiança para melhorar continuamente o código sem medo de quebrar funcionalidades já existentes.

Exemplos de TDD em projetos

Neste tema, vamos explorar **exemplos de TDD aplicados em projetos**, trazendo situações práticas que ilustram como o desenvolvimento orientado por testes se encaixa no dia a dia da construção de software. Ver exemplos concretos nos ajuda a consolidar o entendimento da técnica e a enxergar como o TDD pode ser utilizado em diferentes tipos de sistemas e desafios.

Imagine que estamos desenvolvendo um sistema de cadastro de usuários. Um dos requisitos é que o sistema deve permitir criar um novo usuário

somente se o e-mail informado ainda não estiver registrado. Começamos, então, pelo teste: criamos um método de teste que simula a tentativa de cadastrar um usuário com um e-mail novo e validamos que o cadastro é realizado com sucesso. Em seguida, criamos outro teste que tenta cadastrar um usuário com um e-mail já existente e verificamos que uma exceção é lançada ou que uma mensagem de erro é retornada.

Esses testes definem o comportamento esperado antes mesmo de termos a lógica de cadastro implementada. Após escrever os testes e vê-los falhar (etapa Red), criamos o código mínimo que consulta o repositório de usuários e realiza a verificação. Com os testes passando (etapa Green), podemos refatorar o código para melhorar a clareza ou otimizar a consulta, mantendo a segurança proporcionada pelos testes (etapa Refactor).

Outro exemplo prático pode ser um sistema de vendas onde precisamos calcular o valor total de um carrinho de compras. Começamos escrevendo um teste que cria um carrinho vazio e valida que o total é zero. Depois, criamos um teste que adiciona dois produtos e valida se o total corresponde à soma dos valores dos produtos.

Esses testes guiam a criação da lógica do carrinho, nos forçando a pensar na estrutura necessária: precisamos de uma lista de produtos, de um método para adicionar produtos e de uma forma de calcular a soma dos preços. Assim, ao invés de imaginarmos toda a estrutura de uma vez, vamos construindo o sistema de maneira incremental e guiada pelos requisitos reais.

Em sistemas que interagem com serviços externos, como APIs de pagamento ou serviços de autenticação, o TDD também se aplica. Utilizamos mocks para simular as respostas desses serviços durante os testes. Por exemplo, ao testar um processo de pagamento, criamos um mock do serviço de pagamentos e validamos que, ao receber uma resposta de sucesso, o pedido é confirmado, e que, em caso de falha, o sistema trata o erro corretamente.

O TDD também é extremamente útil em projetos de APIs REST. Antes de implementarmos os endpoints, podemos escrever testes que simulam

requisições HTTP e validam as respostas esperadas: códigos de status, estruturas de retorno, mensagens de erro. Esse tipo de teste orienta a construção da API de forma consistente com as expectativas dos consumidores.

Outro cenário comum de aplicação do TDD é no desenvolvimento de bibliotecas ou componentes reutilizáveis. Ao escrever testes para os comportamentos esperados dos métodos e funções da biblioteca, garantimos que as interfaces sejam claras, que as funcionalidades sejam confiáveis e que futuras alterações não quebrem o contrato estabelecido com os usuários da biblioteca.

Esses exemplos evidenciam que o TDD não é restrito a nenhum tipo específico de aplicação ou tecnologia. Ele pode ser aplicado tanto em sistemas web, quanto em aplicações mobile, APIs, microserviços ou mesmo em módulos internos de sistemas corporativos. O que muda são os detalhes da implementação, mas o princípio permanece o mesmo: escrever o teste primeiro, construir a solução de forma orientada ao comportamento esperado e evoluir o código com segurança e clareza.

Ao nos habituarmos a construir projetos utilizando TDD, mudamos nossa forma de pensar o desenvolvimento: deixamos de focar apenas na implementação para nos concentrar no valor e no comportamento que cada parte do sistema precisa entregar. Nos próximos temas, vamos avançar para estudar os padrões de TDD e como integrar essa prática com ferramentas como JUnit e Mockito, para que possamos aplicar o que estamos aprendendo em projetos ainda mais completos.

Padrões de TDD

Neste tema, vamos conhecer os **padrões de TDD**, que nos ajudam a estruturar melhor nossos testes e a tornar o processo de desenvolvimento ainda mais claro, organizado e sustentável. Seguir padrões no TDD é fundamental para manter a consistência dos testes, facilitar a leitura do código e garantir que o fluxo de Red-Green-Refactor seja aplicado de forma disciplinada em projetos reais.

Um dos primeiros padrões que devemos adotar é a estrutura **Arrange-Act-Assert (AAA)**. Esse padrão organiza o código do teste em três partes bem definidas: primeiro **preparamos o cenário** (Arrange), depois **executamos a ação que queremos testar** (Act) e, por fim, **fazemos as verificações** (Assert). Ao seguir essa estrutura, nossos testes ficam mais fáceis de entender e mais padronizados, permitindo que qualquer pessoa da equipe compreenda rapidamente o que está sendo testado.

Outro padrão importante é o de **nomes descritivos para métodos de teste**. Em vez de nomes genéricos como “testeCadastro” ou “testeValidação”, devemos utilizar nomes que expressem claramente o comportamento que está sendo verificado, como “deveCadastrarUsuarioComEmailValido” ou “deveRejeitarCadastroComEmailDuplicado”. Bons nomes de testes funcionam como uma documentação viva, facilitando o entendimento das funcionalidades testadas.

No TDD, também aplicamos o padrão de **um teste por comportamento**. Cada teste deve validar apenas um aspecto específico da funcionalidade. Evitamos criar testes que verificam múltiplos comportamentos ao mesmo tempo, pois isso dificulta a manutenção, torna a identificação de falhas mais demorada e pode gerar testes frágeis que falham por motivos não relacionados ao objetivo original.

Outro padrão relevante é o de **pequenos ciclos de implementação**. Em vez de escrever grandes blocos de código antes de rodar os testes, trabalhamos em ciclos curtos, criando testes pequenos, implementando apenas o suficiente para fazê-los passar e refatorando continuamente. Esse padrão mantém o processo ágil, facilita a identificação de problemas e garante que estamos sempre trabalhando com um sistema funcional.

Também é importante aplicar o padrão de **refatoração consciente**. Após a etapa Green, não devemos simplesmente melhorar o código aleatoriamente. A refatoração deve ser orientada por princípios de design, como remoção de duplicação, melhoria da legibilidade e simplificação de estruturas complexas. Cada refatoração deve ser pequena e validada imediatamente pelos testes existentes.

No uso de frameworks como JUnit 5 e Mockito, seguimos o padrão de **limpar e organizar o ambiente de teste** antes e depois da execução. Utilizamos anotações como `@BeforeEach` e `@AfterEach` para preparar o cenário de teste e restaurar o estado inicial, garantindo que os testes sejam independentes entre si e que o ambiente esteja sempre previsível.

Outro padrão importante é o de **simular apenas o necessário**. Quando usamos mocks, fakes ou stubs, devemos focar apenas nas dependências que realmente precisam ser simuladas para o teste em questão. Evitamos criar mocks desnecessários, pois isso pode tornar o teste mais complexo, mais difícil de entender e mais frágil diante de mudanças no sistema.

Por fim, um padrão fundamental no TDD é o de **falhar primeiro, corrigir depois**. Antes de começar a implementação, garantimos que o teste realmente falhe, validando que estamos testando um comportamento novo e não simplesmente confirmando algo que já funciona. Só depois disso partimos para o desenvolvimento da solução, seguindo o fluxo disciplinado do Red-Green-Refactor.

Ao adotar esses padrões de TDD, criamos uma base sólida para a prática diária do desenvolvimento orientado por testes. Nossos testes se tornam mais claros, confiáveis e fáceis de manter. O processo de evolução do software se torna mais seguro, organizado e alinhado aos objetivos de negócio.

Nos próximos temas, vamos avançar para a integração do TDD com ferramentas como JUnit e Mockito, explorando como essas práticas se unem para construir testes ainda mais robustos e completos em projetos profissionais.

Integração do TDD com JUnit e Mockito

Neste tema, vamos explorar a **integração do TDD com JUnit e Mockito**, duas ferramentas que, quando utilizadas juntas, potencializam a prática do desenvolvimento orientado por testes. Entender como elas se complementam nos permite construir suítes de testes mais robustas, organizadas e capazes de validar tanto o comportamento interno quanto a

interação entre componentes.

O JUnit 5 é a base da criação e execução dos testes no TDD. Ele nos fornece a estrutura necessária para definirmos os métodos de teste, organizarmos o ciclo de vida dos testes e aplicarmos assertivas para validar os comportamentos esperados. Ao escrevermos o primeiro teste no ciclo Red do TDD, usamos JUnit para estruturar o método de teste e para executar automaticamente a verificação dos resultados.

Quando a funcionalidade em desenvolvimento depende de outros componentes — como repositórios, serviços externos ou integrações —, entramos no campo da necessidade de simulação. É aqui que o Mockito se integra ao processo. Em vez de criar implementações reais para essas dependências (o que poderia tornar os testes lentos e instáveis), utilizamos o Mockito para criar mocks, que são versões simuladas e controladas dessas dependências.

No fluxo do TDD, isso acontece naturalmente: no estágio Red, escrevemos o teste e criamos os mocks necessários para simular o ambiente. No estágio Green, implementamos o comportamento mínimo para fazer o teste passar, utilizando os mocks configurados para retornar as respostas esperadas. E, no estágio Refactor, podemos melhorar tanto o código de produção quanto os próprios testes, ajustando a configuração dos mocks para torná-los mais claros e aderentes ao comportamento esperado.

No JUnit 5, combinamos anotações como `@Test`, `@BeforeEach`, `@AfterEach` e `@DisplayName` para organizar nossos testes e descrever seus propósitos. O Mockito entra com anotações como `@Mock` para criar as dependências simuladas e `@InjectMocks` para injetá-las automaticamente na unidade sob teste. Também usamos métodos como `when().thenReturn()` para configurar o comportamento dos mocks e `verify()` para validar se as interações aconteceram como esperado.

Essa integração entre JUnit e Mockito é especialmente poderosa porque nos permite seguir o TDD mesmo em sistemas complexos, onde as unidades de código raramente são completamente isoladas. Conseguimos focar na lógica principal do componente que estamos desenvolvendo,

simulando todas as dependências ao redor de forma controlada e previsível.

Outro ponto forte da integração entre essas ferramentas é o suporte à execução rápida dos testes. Como os mocks substituem recursos pesados, como bancos de dados ou APIs externas, nossos testes continuam ágeis, permitindo ciclos curtos de Red-Green-Refactor e facilitando o desenvolvimento incremental que o TDD propõe.

Ao estruturarmos nossos projetos com essa integração, também fortalecemos a qualidade dos testes. Podemos testar fluxos de dados, comportamentos condicionais, exceções e interações entre componentes com muito mais controle e precisão. Além disso, criamos uma suíte de testes que serve como documentação viva do sistema, facilitando a compreensão e a manutenção do projeto ao longo do tempo.

Para que a integração entre TDD, JUnit e Mockito seja realmente produtiva, é importante seguir boas práticas, como criar testes pequenos e focados, simular somente o necessário, manter os testes claros e evitar acoplamentos desnecessários entre o código de produção e os detalhes da implementação dos testes.

Ao dominarmos a integração do TDD com JUnit e Mockito, damos um passo importante para atuar em projetos profissionais de maneira mais estruturada, segura e produtiva. Nos próximos temas, vamos aplicar esses conceitos em exemplos mais completos de projetos reais, consolidando tudo o que vimos até aqui em práticas que poderão ser utilizadas no nosso dia a dia de desenvolvimento.

Refatoração inicial baseada em testes

Neste tema, vamos entender o conceito de **refatoração inicial baseada em testes**, uma prática que reforça o papel dos testes como guia para a melhoria contínua do código desde as primeiras fases do desenvolvimento. A refatoração é um dos pilares do ciclo Red-Green-Refactor no TDD e representa o momento em que, após fazer o teste passar, voltamos ao código para aprimorar sua estrutura, sem alterar seu comportamento.

A refatoração inicial acontece logo após conseguirmos que o primeiro teste passe. Como, no estágio Green, escrevemos o código mais simples possível para satisfazer o teste, é natural que a primeira versão da solução seja funcional, mas ainda não seja a melhor em termos de organização, clareza ou qualidade técnica. A refatoração vem para corrigir isso, sempre com a segurança de que os testes garantirão que não estamos quebrando o que já foi validado.

Durante a refatoração, buscamos tornar o código mais legível, modularizar funções, eliminar duplicações, nomear variáveis de forma mais clara e aplicar princípios de design como SOLID. Melhoramos a estrutura interna do código, preparando-o para crescer de maneira sustentável à medida que novas funcionalidades forem adicionadas.

A principal regra da refatoração inicial é: alterar a estrutura sem alterar o comportamento. Os testes que já foram escritos nos dão a confiança necessária para fazer essas mudanças com segurança. Se um teste falhar após a refatoração, é um sinal imediato de que modificamos algo além da estrutura e precisamos revisar as alterações.

No contexto do JUnit 5 e do Mockito, essa prática é muito facilitada. Como conseguimos executar os testes de maneira rápida e automatizada a qualquer momento, podemos fazer pequenas refatorações frequentes, mantendo o código saudável sem precisar de grandes reescritas ou manutenções emergenciais no futuro.

A refatoração inicial baseada em testes também nos ajuda a manter o foco no comportamento do sistema. Não começamos tentando escrever o código mais bonito ou mais sofisticado. Primeiro, garantimos que a funcionalidade exista e esteja correta. Só depois, com a segurança dos testes, aplicamos melhorias estruturais. Essa abordagem incremental é mais produtiva, reduz riscos e gera sistemas de melhor qualidade ao longo do tempo.

Outro benefício importante dessa prática é a criação de um ciclo de desenvolvimento sustentável. Quando incorporamos a refatoração como parte natural do processo, evitamos o acúmulo de dívidas técnicas e

reduzimos a necessidade de grandes intervenções corretivas. O sistema evolui de forma mais contínua, com menos impactos negativos e menos interrupções.

É importante lembrar que a refatoração não é apenas uma ação estética. Ela tem objetivos claros: melhorar a legibilidade, facilitar a manutenção, reduzir a complexidade e preparar o sistema para a próxima funcionalidade. Cada melhoria feita deve ter um propósito prático, sempre validado pelos testes existentes.

Ao dominarmos a prática da refatoração inicial baseada em testes, elevamos nossa capacidade de construir software de alta qualidade de maneira disciplinada e segura. Construimos soluções que não apenas atendem às necessidades atuais, mas que também estão preparadas para evoluir de maneira saudável no futuro.

Com esse tema, encerramos o módulo sobre aplicação prática de TDD, consolidando o ciclo completo de escrever testes primeiro, implementar soluções simples, e evoluir o código com responsabilidade e clareza. A partir daqui, estaremos prontos para avançar para novas práticas de testes em outros contextos de desenvolvimento de software.

Conteúdo Bônus

Recomendamos a leitura do livro *TDD – Test Driven Development na Prática*, de Maurício Aniche, como material complementar para aprofundar o entendimento sobre TDD na prática. A obra oferece uma abordagem prática e aprofundada sobre o TDD, abordando desde os fundamentos da qualidade de software até técnicas avançadas de testes. Explora o modelo de programação TDD, recursos como testes dinâmicos, injeção de dependências, execução paralela e integração com ferramentas como Mockito, Spring, Selenium, Cucumber e Docker. Além disso, apresenta boas práticas para escrever testes significativos e gerenciar atividades de teste em projetos de software em constante evolução. Com exemplos reais e orientações práticas, o livro capacita desenvolvedores e testadores a

aprimorar a qualidade de suas aplicações Java por meio de testes eficazes e bem estruturados.

Referências Bibliográficas

ANICHE, Maurício. ***TDD – Test Driven Development na Prática***. Rio de Janeiro: Ciência Moderna, 2014. ISBN 9788539903270.

Ir para exercício